Computer Vision Project Report

Georg RIEKHAKAINEN, Isabelle LIEW YU CHEN, Norelle CHY-GLASSCOCK, and Thien Anh NGUYEN

Abstract

Image classification is one of the most famous and direct applications of computer vision models. Training a model to recognize the class of an image correctly is associated with challenges, but the final prediction made by the model can easily outperform human performance. In this report, we cover our solution to achieve 97% of accuracy on the test set with fine-tuned Visual Transformer pre-trained by Google (namely, google/vit-base-patch16-224). While this report is intended to describe the fine-tuning process of the pre-trained ViT, the mentioned process is relatively straightforward and does not require a lot of computation. We will also disclose challenges associated with the training of random-weight initialized CNN models, and highlight differences between the two.

Contents

1 Preprocessing **3**
1.1 Challenges and solutions . . . 3
1.1.1 Applied challenges . . . 3
1.1.2 Dataset content challenges . . . 3
1.2 Data Augmentation . . . 4

2 Architecture of the model **5**

3 Metrics of performance **5**
3.1 Training . . . 6
3.2 Evaluation . . . 6

4 Performance **6**
4.1 Training . . . 6
4.2 Validation . . . 6
4.3 Final Performance . . . 7

5 AI and Code **7**

6 Conclusion **8**

1 Preprocessing

1.1 Challenges and solutions

The role of the preprocessing step is undeniable, whether it is a model trained from scratch or one pre-trained by a third-party provider. Most common challenges and choices include: size choice and compliance with input format expected by model, images storage format, size constraints for dataset to be loaded into memory, color channels choice, and, finally, dataset imbalance.

1.1.1 Applied challenges

In our project, we have tried to train both our own CNN model and a fine-tune pre-trained ViT model. With CNN we were free to choose the size our model expects, while the Transformer model was locked to accept only pictures 224x224 as an input. On the contrary, the dataset images varied from 70x70 to 220x220, so one common denominator was used. For example, we tried to reduce all image sizes to 70x70 (for the purpose of increasing speed of training), but later we used the ViTImageProcessor module to conform images to the expected format of ViT. Another problem was format mismatch. Provided images in the training and test folder were stored as PNG (4 channel, RGBA), where A stands for (opacity). The most obvious solution is to drop this parameter and convert images to 3 channels, however we realised that it would mean we change the background from 'transparent' to 'white'. While presumably not a problem for ViT, this made a drastic difference for the CNN model, as the core part of computer vision models is pattern recognition of pixel intensity. As a result, our model was somewhat predicting background patterns and not object patterns. To account for that, we used the formula

$$image_{new} = 1 - image$$

, which intended to highlight dark elements and made light elements of the picture dark instead. Finally, for the CNN model we extracted grayscale form of image, for the reason of maintaining high training speed. The exact ViT Processor flow is as follows: "Images are resized/rescaled to the same resolution (224x224) and normalized across the RGB channels with mean (0.5, 0.5, 0.5) and standard deviation (0.5, 0.5, 0.5)."

1.1.2 Dataset content challenges

One of the things we discovered is that classes in the training dataset were unequally represented. We present the distribution of classes below:

Table 1: Classes distribution in training set

Class	Train	Validation
Apple	1539	385
Google	1501	376
Whatsapp	1315	329
Samsung	1334	333
Mozilla	318	79
Messenger	464	116
Total	7903	1916

Obviously, this pose a problem for training, because models can become overconfident for most occuring classes in the dataset, misclassifying Mozilla and Messenger classes to something else. To prevent this, a few solutions are possible:

- **Find missing samples**: The solution would require extensive action beyond project level, so we decided not to pursue this solution. Even if we did, the whole population of Mozilla emoji is limited to 1035 examples, so we would not be able to represented classes "equally".
- **Cut samples amount to an equal level**: We decided to not follow this solution for the reason of over-simplifying our inputs, given images small size (up to 200 pixels only).
- **Apply weighting to loss during training**: This was our chosen solution. By placing a higher weight on under represented classes (i.e. Mozilla and Messenger), we force the model to learn their features more aggressively, which allowed us to train the model more effectively.

Table 2: Classes weights in training mode

Class	Weight
Apple	0.734
Google	0.752
Whatsapp	0.859
Facebook	0.846
Samsung	0.788
Mozilla	3.550
Messenger	2.433

1.2 Data Augmentation

We tried to use data augmentation in both scenarios, but we did not achieve any improvements. The possible reason might be that the input images are very homogeneous (centered emoji, same background, similar scale), so typical data augmentation techniques were not as helpful.

2 Architecture of the model

In our case, fine-tuning process was represented by adding Dense layer with 7 neurons to the fixed model weights. The ViT model architecture is described as follows :

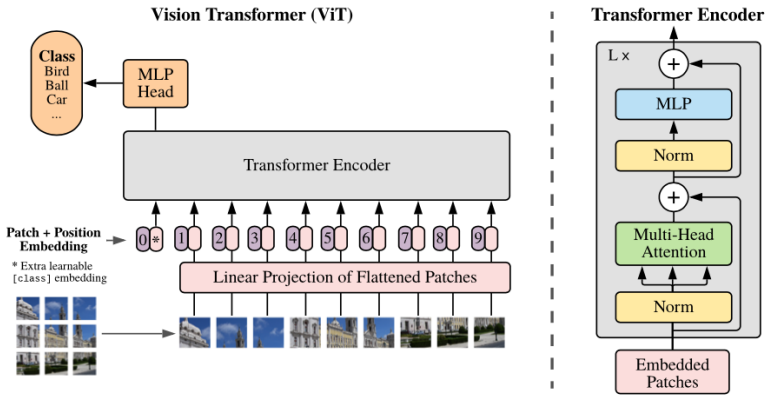


Figure 1: ViT Model Architecture. Source: Github

For the purpose of comparison, CNN model that we tried had an architecture as follows:

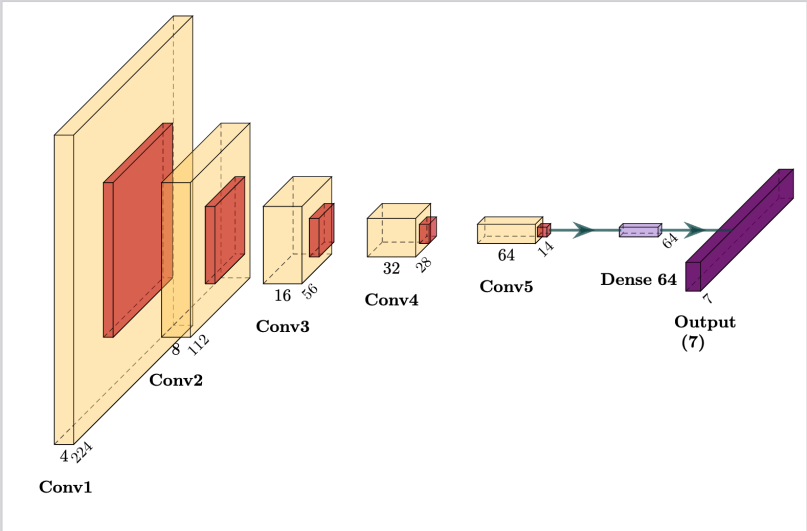


Figure 2: CNN Model Architecture

3 Metrics of performance

We decided to track performance during training and evaluation separately.

3.1 Training

For the training, we were focusing mostly on

$$val_{loss}, val_{accuracy}, val_{accuracy-3}$$

We had the same metric for the training part of training set, but tracking it alone would make us prone to over-confidence and biased for over-fitting. We will present our training results in the next section.

3.2 Evaluation

To track the performance of our model, we evaluated our model with the above-mentioned accuracy metric (which was also a target one in Kaggle dataset), we used

$$sklearn.classification_report()$$

and

$$confusion\ matrix$$

to get a deeper understanding of our model performance. If we were to try different fine-tuning approaches, we would use classification report to compare models between each other.

4 Performance

4.1 Training

Table 3: Training performance across different epochs[1]

Epoch	Val_{loss}	$\text{Val}_{accuracy}$	$\text{Val}_{top3_accuracy}$
0	0.4592	0.9175	0.9808
1	0.2704	0.9479	0.9939
2	0.1978	0.9661	0.9949
3	0.1766	0.9641	0.9965
4	0.1568	0.9717	0.9954

[1] This result is recorded in one of the runs of the notebook, in particular, after formatting it to the submission form. The model that produced results for Kaggle competition had slightly different training performance, but it shared the same magnitude of precision.

4.2 Validation

Actually, we can see that model has achieved a decent level of pattern recognition ability and has a balanced view over both the well-represented and under-represented classes. This is supported by the confusion matrix below.

Table 4: Classification report

Class	Precision	Recall	f1-score	Support
apple	0.956	0.969	0.963	385
google	0.966	0.984	0.975	376
whatsapp	0.975	0.954	0.965	329
facebook	0.979	0.976	0.977	333
samsung	0.983	0.975	0.979	358
mozilla	0.962	0.962	0.962	79
messenger	0.983	0.974	0.978	116

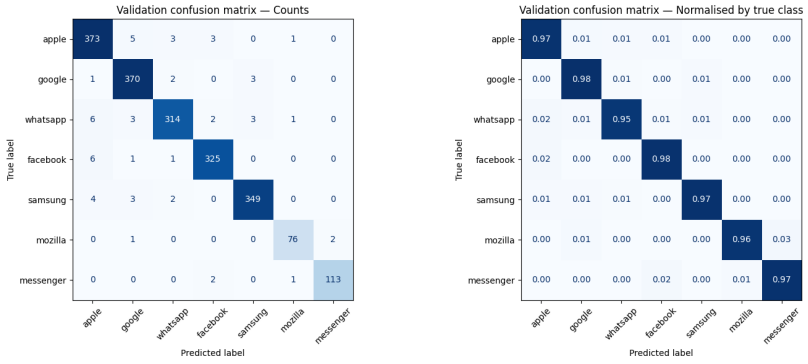


Figure 3: Confusion matrix

4.3 Final Performance

In the final (test) evaluation, contents of which are not disclosed, our model has been able to achieve high score of

$$0.96396/1$$

on 4,500 pictures (half of test dataset, consider it to be *validation* subset during the competition). The final and definitive (private) score is

$$0.97165/1.0$$

.

5 AI and Code

The final notebook was reorganised, cleaned up and documented with the help of Claude (Anthropic), starting from our own experimental notebook. The prompts used are listed in the "AI usage" section at the top of the notebook. The experiments, modelling decisions and the report are our own work.

- **Link to Github repo**: Available here
- **Link to final notebook**: Available here

- **Link to exploratory notebook**: Available here

6 Conclusion

This report has summarized our team's efforts to achieve the best score possible on predicting the class of emoji given multiple constraints, some of which were not disclosed in the competition. In the end of our project, we were able to explore the problem constraints, try different models architectures, choose relevant metrics, and finally achieve 97% of accuracy, mispredicting less than 360 emojis from the test data.